

C Programming Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This C Programming course is designed to build a strong foundation in programming and problem-solving using the C language. The course starts with programming fundamentals and gradually progresses to functions, arrays, strings, pointers, structures, file handling, dynamic memory allocation, data structures, debugging, and practical project development.

The syllabus is suitable for beginners, students, engineering graduates, working professionals, and anyone who wants to strengthen core programming concepts before moving to C++, Java, Python, embedded systems, operating systems, or data structures and algorithms.

Course Objectives

By the end of this course, learners will be able to:

- Understand the fundamentals of computer programming.
- Write, compile, execute, and debug C programs.
- Work with variables, constants, operators, and expressions.
- Use decision-making and looping constructs.
- Design modular programs using functions.
- Work with arrays, strings, pointers, structures, and unions.
- Understand memory management and dynamic memory allocation.
- Perform file input/output operations.
- Implement basic data structures using C.
- Solve programming problems using structured programming techniques.
- Build console-based mini projects in C.
- Develop a strong foundation for Data Structures and Algorithms.

Module 1: Introduction to Programming and C

Topics

- Introduction to Computer Programming
- What is a Programming Language?
- Low-Level, Middle-Level, and High-Level Languages
- Compiler, Interpreter, Assembler, and Linker
- Source Code, Object Code, and Executable Files
- History and Evolution of C
- Features and Applications of C
- Why Learn C?
- C Standards
- K&R C
- ANSI C / C89 / C90
- C99
- C11
- C17
- C23 Overview
- Structure of a C Program
- Character Set
- Tokens in C
- Keywords
- Identifiers

- Constants
- Variables
- Comments
- Header Files
- Preprocessor Directives
- `main()` Function
- Return Values
- Writing the First C Program
- Compilation and Execution Process

Practical Exercises

- Hello World Program
- Display Personal Information
- Basic Arithmetic Program
- Compile and Run Programs Using GCC

Module 2: Setting Up the C Development Environment

Topics

- Installing a C Compiler
- GCC and Clang Overview
- IDEs and Editors
- Visual Studio Code
- Code::Blocks
- Dev-C++
- CLion
- Online Compilers
- Creating and Saving C Source Files
- Compiling from Command Line
- GCC Compilation Commands
- Compilation Stages
- Preprocessing
- Compilation
- Assembly
- Linking
- Common Compilation Errors
- Syntax Errors
- Runtime Errors
- Logical Errors
- Debugging Basics
- Using Terminal / Command Prompt
- Introduction to Makefiles

Module 3: Variables, Constants, and Data Types

Topics

- Variables in C
- Rules for Naming Variables
- Variable Declaration
- Variable Initialization
- Constants
- Literal Constants
- Symbolic Constants

- const Keyword
- #define
- Fundamental Data Types
- char
- int
- float
- double
- void
- Type Modifiers
- short
- long
- signed
- unsigned
- Integer Types
- Floating-Point Types
- Character Types
- Boolean Values in C
- _Bool and stdbool.h
- Size of Data Types
- sizeof Operator
- Range of Data Types
- Format Specifiers
- Escape Sequences
- Type Conversion
- Implicit Conversion
- Explicit Type Casting
- Scope and Lifetime Overview

Practical Exercises

- Calculate Area and Perimeter
- Temperature Conversion
- Simple Interest Calculator
- Swap Two Numbers
- Display Size of Different Data Types

Module 4: Input and Output in C

Topics

- Standard Input and Output
- printf()
- scanf()
- Format Specifiers
- Field Width
- Precision
- Escape Sequences
- Character Input and Output
- getchar()
- putchar()
- String Input and Output
- fgets()
- puts()
- Limitations of scanf()
- Buffering Basics

- Input Validation Basics

Practical Exercises

- Read and Display Student Details
- Arithmetic Calculator
- Read Character and Display ASCII Value
- Convert Character Case
- Formatted Output Programs

Module 5: Operators and Expressions

Topics

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Increment and Decrement Operators
- Bitwise Operators
- AND
- OR
- XOR
- NOT
- Left Shift
- Right Shift
- Conditional / Ternary Operator
- sizeof Operator
- Comma Operator
- Operator Precedence
- Operator Associativity
- Expressions
- Lvalue and Rvalue Concepts
- Type Conversion in Expressions
- Short-Circuit Evaluation

Practical Exercises

- Even or Odd Number
- Maximum of Two Numbers
- Maximum of Three Numbers
- Bitwise Operations
- Check Leap Year

Module 6: Decision-Making Statements

Topics

- Introduction to Control Flow
- if Statement
- if-else
- Nested if
- else-if Ladder
- Multiple Conditions
- Logical Expressions
- switch Statement
- case

- default
- break
- Nested switch
- When to Use if vs switch
- Conditional Operator

Practical Exercises

- Grade Calculator
- Largest of Three Numbers
- Positive, Negative, or Zero
- Vowel or Consonant
- Menu-Driven Calculator
- Electricity Bill Calculator

Module 7: Looping Statements

Topics

- Need for Iteration
- while Loop
- do-while Loop
- for Loop
- Nested Loops
- Infinite Loops
- Loop Control Statements
- break
- continue
- goto Statement
- Best Practices for Loops

Practical Exercises

- Print Numbers in a Range
- Multiplication Table
- Sum of Natural Numbers
- Factorial
- Fibonacci Series
- Reverse a Number
- Palindrome Number
- Armstrong Number
- Prime Number
- Prime Numbers in a Range
- Number Patterns
- Star Patterns

Module 8: Functions

Topics

- Introduction to Functions
- Benefits of Functions
- Function Declaration
- Function Definition
- Function Call
- Function Prototype
- Return Types

- Function Parameters
- Arguments
- Categories of Functions
- No Arguments, No Return Value
- Arguments, No Return Value
- No Arguments, Return Value
- Arguments and Return Value
- Actual and Formal Parameters
- Call by Value
- Simulating Call by Reference Using Pointers
- Local Variables
- Global Variables
- Scope
- Lifetime
- Storage Classes
- auto
- register
- static
- extern
- Recursive Functions
- Recursion vs Iteration
- Inline Functions Overview

Practical Exercises

- Function-Based Calculator
- Prime Number Using Function
- Factorial Using Recursion
- Fibonacci Using Recursion
- GCD and LCM Using Functions
- Swap Values Using Function

Module 9: Arrays

Topics

- Introduction to Arrays
- Need for Arrays
- One-Dimensional Arrays
- Array Declaration
- Array Initialization
- Accessing Array Elements
- Traversing Arrays
- Reading and Writing Arrays
- Array Indexing
- Passing Arrays to Functions
- Searching in Arrays
- Sorting Arrays
- Two-Dimensional Arrays
- Matrix Representation
- Multi-Dimensional Arrays
- Array Memory Layout
- Variable Length Arrays Overview

Algorithms and Practical Exercises

- Sum and Average of Array Elements
- Find Maximum and Minimum
- Reverse an Array
- Linear Search
- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Matrix Addition
- Matrix Subtraction
- Matrix Multiplication
- Matrix Transpose
- Diagonal Elements

Module 10: Strings

Topics

- Strings in C
- Character Arrays
- Null Character
- String Declaration
- String Initialization
- Reading Strings
- Writing Strings
- Standard String Functions
- `strlen()`
- `strcpy()`
- `strncpy()`
- `strcat()`
- `strncat()`
- `strcmp()`
- `strncmp()`
- `strchr()`
- `strstr()`
- String Manipulation Without Library Functions
- Array of Strings
- Passing Strings to Functions
- String Input Safety
- Common String Errors

Practical Exercises

- Find String Length
- Reverse a String
- Check Palindrome String
- Count Vowels and Consonants
- Count Words
- Compare Two Strings
- Copy String Without `strcpy()`
- Concatenate Strings
- Character Frequency

Module 11: Pointers

Topics

- Introduction to Memory Addresses
- Pointer Variables
- Declaring Pointers
- Initializing Pointers
- Address Operator &
- Dereference Operator *
- Pointer Arithmetic
- Pointers and Arrays
- Arrays of Pointers
- Pointer to Pointer
- Pointers and Strings
- Passing Pointers to Functions
- Call by Reference Using Pointers
- Pointer to Function
- Function Pointers
- void Pointers
- NULL Pointers
- Dangling Pointers
- Wild Pointers
- Constant Pointers
- Pointer to Constant
- Common Pointer Errors

Practical Exercises

- Swap Two Numbers Using Pointers
- Traverse Array Using Pointer
- String Operations Using Pointers
- Pointer Arithmetic Demonstrations
- Function Pointer Example

Module 12: Dynamic Memory Allocation

Topics

- Static vs Dynamic Memory
- Stack and Heap
- Dynamic Memory Allocation
- `malloc()`
- `calloc()`
- `realloc()`
- `free()`
- Memory Leaks
- Dangling Pointers
- Double Free
- Memory Allocation Failures
- Checking NULL After Allocation
- Dynamic Arrays
- Dynamic Strings
- Dynamic 2D Arrays Overview

Practical Exercises

- Dynamically Allocate Integer Array
- Resize an Array Using `realloc()`

- Dynamic Student Records
- Dynamic Matrix Allocation

Module 13: Structures

Topics

- Introduction to Structures
- Structure Declaration
- Structure Variables
- Initializing Structures
- Accessing Members
- Dot Operator
- Array of Structures
- Nested Structures
- Passing Structures to Functions
- Returning Structures from Functions
- Pointer to Structure
- Arrow Operator
- Self-Referential Structures
- typedef with Structures
- Memory Padding and Alignment Overview

Practical Exercises

- Student Record Management
- Employee Record System
- Product Inventory
- Library Book Records
- Structure with Nested Address Details

Module 14: Unions and Enumerations

Topics

- Union Declaration
- Union Variables
- Structure vs Union
- Memory Allocation in Union
- Accessing Union Members
- Anonymous Unions Overview
- Enumerations
- enum
- Named Constants Using Enum
- typedef
- Use Cases of Enum

Practical Exercises

- Employee Data Using Union
- Menu Options Using Enum
- Compare Structure and Union Memory Usage

Module 15: Preprocessor Directives and Macros

Topics

- C Preprocessor

- `#include`
- `#define`
- Symbolic Constants
- Object-Like Macros
- Function-Like Macros
- Macro Arguments
- Nested Macros
- `#undef`
- Conditional Compilation
- `#if`
- `#ifdef`
- `#ifndef`
- `#else`
- `#elif`
- `#endif`
- Predefined Macros
- `__FILE__`
- `__LINE__`
- `__DATE__`
- `__TIME__`
- Header Guards
- Macro Pitfalls
- Inline Functions vs Macros

Practical Exercises

- Create Mathematical Macros
- Conditional Debug Logging
- Header Guard Example
- Platform-Specific Compilation Example

Module 16: File Handling

Topics

- Introduction to Files
- File Streams
- FILE Pointer
- Opening Files with `fopen()`
- File Modes
- Read
- Write
- Append
- Binary Modes
- Closing Files with `fclose()`
- Character I/O
- `fgetc()`
- `fputc()`
- String I/O
- `fgets()`
- `fputs()`
- Formatted File I/O
- `fprintf()`
- `fscanf()`
- Binary File I/O

- `fread()`
- `fwrite()`
- File Positioning
- `fseek()`
- `ftell()`
- `rewind()`
- End-of-File Detection
- Error Handling
- `feof()`
- `ferror()`
- `perror()`
- Renaming and Deleting Files
- Random Access Files

Practical Exercises

- Write Data to Text File
- Read Data from Text File
- Copy One File to Another
- Count Characters, Words, and Lines
- Student Database Using Files
- Binary File Record Management

Module 17: Command-Line Arguments

Topics

- `argc`
- `argv`
- Command-Line Arguments
- Passing Values from Terminal
- Converting String Arguments to Numbers
- `atoi()` Overview
- Safer Alternatives Such as `strtol()`
- Building Command-Line Utilities

Practical Exercises

- Add Numbers from Command Line
- File Copy Utility
- Command-Line Calculator

Module 18: Error Handling and Debugging

Topics

- Types of Programming Errors
- Syntax Errors
- Compilation Errors
- Linking Errors
- Runtime Errors
- Logical Errors
- Debugging Techniques
- Print Debugging
- Using Debuggers
- GDB Basics
- Breakpoints

- Step Into / Step Over
- Inspecting Variables
- Stack Traces
- Assertions
- `assert.h`
- `errno`
- `perror()`
- Defensive Programming
- Input Validation
- Common C Programming Bugs
- Buffer Overflow
- Array Out-of-Bounds Access
- Null Pointer Dereference
- Uninitialized Variables
- Memory Leaks
- Use-After-Free

Module 19: Bit Manipulation

Topics

- Binary Number System Review
- Bitwise AND
- Bitwise OR
- Bitwise XOR
- Bitwise NOT
- Left Shift
- Right Shift
- Bit Masks
- Setting a Bit
- Clearing a Bit
- Toggling a Bit
- Checking a Bit
- Applications in Embedded Systems

Practical Exercises

- Check Odd/Even Using Bitwise Operator
- Swap Without Temporary Variable
- Set/Clear/Toggle Specific Bits
- Count Set Bits
- Check Power of Two

Module 20: Data Structures Using C

Topics

Linked Lists

- Introduction to Linked Lists
- Node Creation
- Singly Linked List
- Insertion
- Deletion
- Traversal
- Searching
- Reversing Linked List

- Doubly Linked List Overview
- Circular Linked List Overview

Stacks

- Stack Concept
- Array-Based Stack
- Linked-List-Based Stack
- Push
- Pop
- Peek
- Applications of Stack

Queues

- Queue Concept
- Array-Based Queue
- Linked-List-Based Queue
- Enqueue
- Dequeue
- Circular Queue
- Priority Queue Overview

Trees

- Introduction to Trees
- Binary Trees
- Binary Search Trees
- Tree Traversals
- Preorder
- Inorder
- Postorder

Searching and Sorting

- Linear Search
- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort Overview
- Quick Sort Overview

Module 21: Advanced C Concepts

Topics

- Memory Layout of a C Program
- Text Segment
- Data Segment
- BSS
- Heap
- Stack
- Scope and Linkage
- Internal and External Linkage
- `static` Keyword in Depth
- `extern` Keyword
- `volatile` Keyword
- `restrict` Keyword Overview

- const Correctness
- Function Pointers
- Callback Functions
- Pointer to Functions
- Arrays of Function Pointers
- Flexible Array Members
- Compound Literals
- Designated Initializers
- Bit Fields
- Endianness
- Undefined Behavior
- Implementation-Defined Behavior
- Unspecified Behavior
- Portability Considerations

Module 22: Modular Programming

Topics

- Breaking Programs into Modules
- Header Files
- Source Files
- Function Declarations
- Include Guards
- Separate Compilation
- Linking Multiple Source Files
- Reusable Libraries
- Static Libraries Overview
- Shared Libraries Overview
- Makefiles
- Project Folder Organization

Practical Exercise

Build a multi-file C application with:

- main.c
- Header files
- Utility functions
- Data access module
- Build automation using Makefile

Module 23: C Standard Library Overview

Important Header Files

- <stdio.h>
- <stdlib.h>
- <string.h>
- <ctype.h>
- <math.h>
- <time.h>
- <stdbool.h>
- <stdint.h>
- <limits.h>
- <float.h>
- <assert.h>

- `<errno.h>`

Topics

- Standard I/O Functions
- String Functions
- Character Classification Functions
- Mathematical Functions
- Random Number Generation
- Date and Time Functions
- Integer Types
- Limits and Ranges
- Utility Functions
- Searching and Sorting Using `qsort()` and `bsearch()`

Module 24: Secure and Professional C Programming Practices

Topics

- Writing Readable Code
- Naming Conventions
- Code Indentation
- Modular Design
- Avoiding Magic Numbers
- Proper Error Handling
- Safe String Handling
- Preventing Buffer Overflow
- Avoiding Undefined Behavior
- Proper Memory Management
- Avoiding Memory Leaks
- Checking Return Values
- Input Validation
- Defensive Programming
- Code Reviews
- Compiler Warnings
- Recommended GCC Flags
- Static Analysis Overview
- Sanitizers Overview
- AddressSanitizer
- UndefinedBehaviorSanitizer
- Introduction to Valgrind

Module 25: Problem-Solving and Algorithmic Thinking

Topics

- Understanding the Problem
- Identifying Inputs and Outputs
- Breaking Problems into Steps
- Algorithms
- Flowcharts
- Pseudocode
- Dry Run
- Trace Tables
- Time Complexity Introduction
- Space Complexity Introduction

- Big-O Notation Basics
- Choosing Efficient Solutions
- Debugging Logical Problems

Practice Categories

- Number-Based Problems
- Pattern Problems
- Array Problems
- String Problems
- Matrix Problems
- Pointer Problems
- Recursion Problems
- File Handling Problems
- Data Structure Problems

Module 26: Mini Projects

Students should complete multiple mini projects such as:

- Calculator Application
- Student Grade Management System
- Employee Record Management System
- Library Management System
- Contact Management System
- Banking Management System
- Quiz Application
- Inventory Management System
- Tic-Tac-Toe Game
- Number Guessing Game
- Expense Tracker
- Railway Reservation Simulation
- File-Based Address Book
- Simple Text Editor
- Command-Line To-Do Application

Module 27: Capstone Project

Learners will build one complete project using multiple concepts from the course.

Suggested Capstone Projects

- Library Management System
- Hospital Management System
- Student Information System
- Banking Management System
- Inventory and Billing System
- Employee Payroll System
- Hotel Reservation System
- File-Based Database Application

Capstone Requirements

- Modular source code
- Functions
- Structures
- Pointers
- Dynamic memory allocation

- File handling
- Input validation
- Error handling
- Search and sorting
- Menu-driven interface
- Documentation
- Git-based version control

Module 28: Git and GitHub for C Developers

Topics

- Introduction to Version Control
- Installing Git
- Creating a Repository
- `git init`
- `git add`
- `git commit`
- `git status`
- `git log`
- Branching Basics
- GitHub Repository Creation
- Push and Pull
- README Documentation
- `.gitignore`
- Building a Programming Portfolio

Module 29: AI-Assisted C Development

Topics

- Using AI Coding Assistants Responsibly
- Generating Boilerplate Code
- Explaining Existing C Code
- Debugging with AI Assistance
- Generating Test Cases
- Understanding Compiler Errors
- Refactoring Suggestions
- Code Review with AI
- Security Risks of Blindly Using Generated Code
- Validating AI-Generated Programs
- Prompting Techniques for Programming Help

Recommended Practice

Students should learn to:

- Ask AI for explanations rather than only solutions.
- Verify generated code manually.
- Compile with warnings enabled.
- Test edge cases.
- Understand every generated line before submission.

Module 30: Interview Preparation

C Programming Interview Topics

- Variables and Data Types

- Storage Classes
- Operators
- Arrays
- Strings
- Functions
- Recursion
- Pointers
- Pointer Arithmetic
- Arrays vs Pointers
- Structures and Unions
- Dynamic Memory Allocation
- File Handling
- Preprocessor
- `const`, `static`, `extern`, and `volatile`
- Function Pointers
- Memory Layout
- Stack vs Heap
- Undefined Behavior
- Data Structures
- Searching and Sorting

Interview Activities

- Conceptual Questions
- Output-Based Questions
- Debugging Questions
- Coding Challenges
- Whiteboard Programming
- Mock Interviews
- Timed Coding Exercises

Recommended Lab Assignments

Learners should complete at least:

- 20 Basic Syntax and Operator Programs
- 20 Conditional and Looping Programs
- 15 Function and Recursion Programs
- 20 Array and Matrix Programs
- 20 String Programs
- 15 Pointer Programs
- 10 Structure and Union Programs
- 10 Dynamic Memory Programs
- 10 File Handling Programs
- 10 Data Structure Programs
- 5 Mini Projects
- 1 Capstone Project

Suggested Course Duration

A complete C Programming course can be delivered in approximately:

- Section — Suggested Duration
- Programming Fundamentals — 6–8 Hours
- C Basics and Operators — 8–10 Hours
- Decision Making and Loops — 8–10 Hours

- Functions and Recursion — 8–10 Hours
- Arrays and Strings — 12–15 Hours
- Pointers and Memory — 12–15 Hours
- Structures and File Handling — 10–12 Hours
- Data Structures — 15–20 Hours
- Advanced C — 8–10 Hours
- Projects and Interview Preparation — 15–20 Hours

Recommended Total Duration: approximately **100–130 hours**, depending on the depth of practical work.

Prerequisites

No prior programming experience is required.

Basic familiarity with computers, files, folders, and command-line usage is helpful.

Tools and Technologies

- C Programming Language
- GCC / Clang Compiler
- Visual Studio Code
- Code::Blocks or CLion
- GDB Debugger
- Git
- GitHub
- Make
- Valgrind
- AddressSanitizer
- Linux / macOS / Windows Command Line

Learning Outcomes

After completing the course, learners should be able to:

- Write structured and modular C applications.
- Compile and debug programs independently.
- Apply functions, arrays, strings, pointers, structures, and files effectively.
- Understand how memory works in low-level programming.
- Build applications using dynamic memory allocation.
- Implement common data structures.
- Write safer and more maintainable C code.
- Use Git and GitHub for project management.
- Solve programming and interview problems using C.
- Progress confidently to C++, embedded systems, operating systems, microcontrollers, or Data Structures and Algorithms.

Recommended Progression After C Programming

After completing C, learners can continue with:

- Data Structures and Algorithms
- C++ Programming
- Object-Oriented Programming
- Operating Systems
- Linux System Programming
- Embedded C
- Microcontroller Programming
- Computer Networks

- Database Programming
- Competitive Programming

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.

ARCHER INFOTECH PUNE
archerinfotech.in